Session 23 — Study Guide

Status: PREPARED DRAFT

What we learned today

1. The App Router's own file-based routing: a folder is a real URL segment, `page.tsx` is that segment's own UI, `layout.tsx` wraps every segment beneath it.
2. **Server Components vs. Client Components** — every component is a real Server Component by default; `"use client"` opts a component into running in the browser, needed only for real interactivity.
3. Dynamic route segments (`[id]`) and this exact Next.js version's own real `params` Promise.
4. `loading.tsx` / `not-found.tsx` / `error.tsx` — three distinct, file-based conventions.
5. A real environment variable for the frontend's own API base URL.
6. The real backend's own real public/protected split — confirmed by reading the actual controllers, not assumed.

Why we needed it

The B4F Board frontend did not exist before today. Every real feature from here on (Tailwind, auth, mutations, comments) needs a real place to live — the App Router's own routing and rendering model is that place.

First-Time Tool Box

Tool: Next.js App Router (this cohort's first real exposure).

What it is: a file-system-based router built into Next.js — folders under `src/app/` become real URL segments.

Why we use it here: it is the real, current, official way to build a Next.js frontend; B4F Board's own final target (`final-preview/b4f-board/`) is built on it too.

Where/how you run it: `npm run dev` (development, with Turbopack and fast refresh) or `npm run build && npm run start` (a real production build) inside `web/` .

Commands/actions needed today:

1. `npx create-next-app@15.5.25 web --typescript --app --no-tailwind --eslint --src-dir --import-alias "@/*" --use-npm`
2. `npm run dev`
3. Creating a folder (e.g. `workspaces/[id]/`) to create a real route.
4. Creating `page.tsx` / `loading.tsx` / `not-found.tsx` / `error.tsx` inside that folder.
5. `npx tsc --noEmit` to check types.
6. `npm run build` to see the real route table (`○` static vs. `ƒ` dynamic).

How to tell it worked: `curl` /the browser shows real backend data at the real URL matching your folder structure.

Common first-time mistakes: forgetting `await params` (this version requires it); writing `"use client"` out of habit on a page that never needed browser interactivity.

The problem, before the solution

Before today, B4F Board was a backend with no frontend at all. Real data existed (workspaces, projects, issues, comments, activity entries), but no real human-facing page could show any of it.

The solution, step by step

Server Components fetch real data directly. WHAT IS IT: an `async` component that `await fetch()`s inside its own body. WHY: no client-side loading state to hand-manage; the request runs before any HTML reaches the browser. WHAT BREAKS WITHOUT IT: nothing breaks exactly, but you would ship real, unnecessary JavaScript to the browser for logic that never needed to leave the server. HOW: `const res = await apiFetch("/workspaces"); const data = await res.json();` WHAT TO LOOK AT: `src/app/workspaces/page.tsx`. WHAT TO EXPECT: real backend data rendered server-side, visible in the initial HTML.

Dynamic route segments + the real `params` Promise. WHAT IS IT: a folder named `[id]` captures any real value at that URL position. WHY: one real file serves every real workspace/project/issue id, instead of one file per id. WHAT BREAKS WITHOUT IT: `params.id` read without `await` is `undefined` at runtime in this exact Next.js version. HOW: `const { id } = await params;` before reading `id` anywhere. WHAT TO LOOK AT: `src/app/workspaces/[id]/page.tsx`. WHAT TO EXPECT: the same file renders a different real workspace depending on the real URL.

`loading` / `not-found` / `error` — three distinct real conventions. WHAT IS IT: three file names Next.js wires up automatically per route segment. WHY: a real pending fetch, a real missing resource, and a real unexpected crash are three genuinely different situations, each needing its own real UI. WHAT BREAKS WITHOUT IT: without `not-found.tsx`, a real missing id shows Next.js's own generic page instead of this app's own message; without `error.tsx` (which MUST be a Client Component), an unexpected crash shows Next.js's own generic error overlay. HOW: create the file, next to `page.tsx`, with the exact name. WHAT TO LOOK AT: `src/app/workspaces/[id]/loading.tsx` / `not-found.tsx` / `error.tsx`. WHAT TO EXPECT: the right real UI for the right real situation, with zero wiring in `page.tsx` itself.

Connecting to earlier sessions

The real backend (Sessions 19-22) already exists; today only builds the real frontend that talks to it. `fetch` itself is not new — PowerUp already used it; what is genuinely new is doing it inside a real Server Component, with no `useEffect` / `useState` for loading at all.

Glossary (required)

- **Server Component:** a real component that runs only on the server; the default for every component under `app/` unless marked otherwise.
- **Client Component:** a real component marked `"use client"`; runs in the real browser, needed for real interactivity (state, events, effects).

- **Dynamic route segment:** a folder like `[id]` that captures any real value at that URL position.
- **App Router:** the file-system-based routing system this app uses (`src/app/`).

Common mistakes

- Reading `params.id` without `await` — reads `undefined` at runtime in this exact version.
- Checking `res.status === 404` instead of `!res.ok` — misses every other real non-2xx case (see the real homework's own STRETCH).
- Fetching every real resource and filtering client-side when the real backend already supports a real query-param filter (`GET /issues?projectId=`).

Common errors and how to debug them

- **"Cannot read properties of undefined"** on a field like `workspace.name`: almost always means the real `fetch` response was never checked for `.ok` / `404` before being used — add the real guard back.
- **A real page renders "undefined" instead of real data:** the real backend returned a non-2xx response (often a real `400` for a malformed id) that was never checked — read the real response body with `curl` first to see what actually came back.
- **Next.js's own generic error overlay, not this app's own UI:** `error.tsx` is missing `"use client"`, or is missing entirely for this route segment.

Never "fix" any of these by wrapping the whole page in a blind `try/catch` that swallows the real error — read the real response/error first.

Review questions (required)

1. Why must `error.tsx` be a Client Component, specifically?
2. What is the real, structural difference between `loading.tsx` and `not-found.tsx` triggering?
3. Why does Workspace Detail fetch every real project and filter it itself, while Project Detail fetches issues with a real `?projectId=` filter instead?
4. Why does `assigneeId` / `reporterId` on Issue Detail show a raw real id instead of a name, this session?

Answers

1. A real error boundary needs real browser-side lifecycle/interactivity (`reset()`, a real `onClick`) that a Server Component cannot provide.
2. `loading.tsx` is automatic, while the real fetch is pending; `not-found.tsx` only renders when the real code explicitly calls `notFound()`.
3. `GET /projects` has no real `workspaceId` filter on the real backend; `GET /issues` already has a real `projectId` filter — using what the real backend actually supports, not reinventing it.
4. `GET /users/:id` genuinely requires a real auth token (Session 25); showing a fake name would be dishonest.

What you should be able to explain now (required)

- The real difference between a Server Component and a Client Component, and why `error.tsx` specifically needs the latter.
- How a dynamic route segment resolves a real URL value into `params`, and why `await` is required in this exact version.
- Why checking a real backend's own actual filter support matters more than assuming a pattern that worked on a different real endpoint.

Not yet (required)

- Tailwind/styling (Session 24).
- Real frontend authentication/tokens (Session 25).
- Redux Toolkit/real shared mutation state (Session 26).
- Comment, mention, and history interfaces (required in Session 27); Notifications UI (outside the current five-session target).